

Laporan Proyek Akhir

SAFEPASS: PASSWORD MANAGER TERENKRIPSI

Disusun untuk memenuhi projek akhir pada mata kuliah Kriptografi

Dosen Pengampu: Lia Farhatuaini, S.Kom., M.Cs.



Disusu Oleh Kelompok SafePass:

Abdullah Fattah	(2488010067)
Muhammad Rizky Saputra	(2488010071)
Najwa Ramadhan	(2488010001)

PROGRAM STUDI INFORMATIKA
FAKULTAS ILMU TARBIYAH DAN KEGURUAN
UIN SIBER SYEKH NURJATI CIREBON
2026 / 1447 H

A. Pendahuluan

Di era digital saat ini, hampir setiap orang memiliki banyak akun online, seperti media sosial, email, layanan keuangan, dan berbagai platform lainnya. Kondisi ini mengharuskan dan membuat pengguna harus mengingat sekaligus mengelola banyak password sekaligus. Namun, pada praktiknya, banyak pengguna menggunakan password yang sama di berbagai akun atau menggunakan password yang lemah agar mudah di ingat. Hal ini meningkatkan resiko kebocoran data, terutama jika salah satu layanan mengalami kendala atau pelanggaran keamanan.

Permasalahan utama yang ingin diselesaikan dalam proyek ini adalah bagaimana menyimpan dan mengelola password dengan aman tanpa mengorbankan kemudahan penggunaan. Untuk itu, kami membuat sebuah sistem bernama **SafePass**, yaitu password manager yang menyimpan seluruh data password dalam bentuk terenkripsi.

Tujuan dari sistem ini adalah :

1. Menyediakan tempat penyimpanan password yang aman
2. Mencegah penggunaan password yang lemah dan berulang
3. Mengimplementasikan konsep kriptografi secara nyata dalam aplikasi

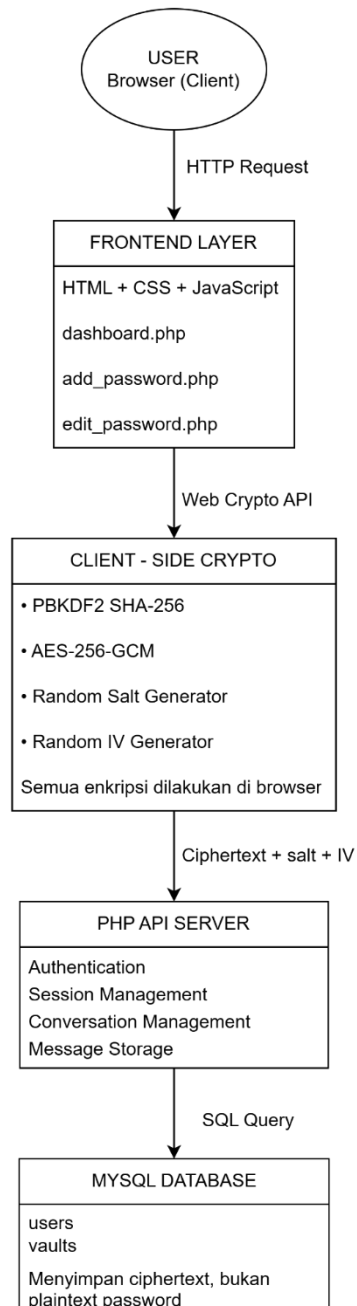
SavePass dirancang menggunakan konsep **zero-knowledge**, yaitu server tidak memiliki akses terhadap isi pengguna data . Dengan demikian, meskipun database server mengalami kebocoran, data password tetap tidak dapat dibaca oleh pihak lain.

Sistem ini ditujukan untuk pengguna umum yang memiliki banyak akun digital dan membutuhkan solusi untuk menyimpan serta mengelola password dengan aman.

B. Arsitektur Sistem

Arsitektur sistem SafePass dirancang dengan pendekatan *client-side encryption*, yaitu memisahkan secara tegas antara proses kriptografi dan penyimpanan data. Pada sistem ini, seluruh proses sensitif seperti enkripsi, dekripsi, dan pembentukan kunci dilakukan di sisi client (browser), sedangkan server hanya berperan sebagai media penyimpanan data dalam bentuk terenkripsi (*ciphertext*). Pendekatan ini dipilih untuk meningkatkan keamanan data pengguna, khususnya dalam mencegah kebocoran informasi sensitif akibat kompromi server.

1. Diagram Arsitektur



2. Komponen Sistem

a. Client Browser

Client menjadi bagian utama dalam keamanan SafePass karena seluruh proses kriptografi dilakukan di sisi client menggunakan JavaScript dan Web Crypto API, seperti PBKDF2 dan AES-GCM. Data asli (plaintext) hanya berada di browser pengguna sebelum dikirim ke server.

b. Server (PHP)

Server berfungsi untuk mengelola autentikasi, layanan aplikasi, dan penyimpanan data. Server tidak melakukan proses enkripsi maupun dekripsi sehingga tidak memiliki akses terhadap isi data pengguna.

c. Database (MySQL)

Database digunakan untuk menyimpan data pengguna dan data vault dalam bentuk ciphertext. Password asli dan master password tidak disimpan sehingga data tetap aman meskipun database diakses pihak lain.

d. Konsep Zero-Knowledge

SafePass menerapkan konsep Zero-Knowledge, yaitu server tidak mengetahui isi password maupun kunci enkripsi pengguna. Semua proses kriptografi dilakukan di sisi client sehingga pengguna memiliki kontrol penuh terhadap data mereka.

3. Alur Data

a. Proses Registrasi

1. User menginput username dan master password
2. Browser membuat salt random
3. Browser menjalankan PBKDF2 SHA-256 sebanyak 100000 iterasi
4. Browser menghasilkan password hash
5. Hash dan salt dikirim ke backend
6. Backend menyimpan data ke database

b. Proses Login

1. User memasukkan username dan master password
2. Backend mengirim salt dan iteration
3. Browser menjalankan PBKDF2 ulang
4. Hash dibandingkan dengan database
5. Server menyimpan ciphertext ke database

c. Proses Penyimpanan Vault

1. User menambah data website, username, password, dan notes
2. Browser membentuk AES key dari master password
3. Data dienkripsi menggunakan AES-256-GCM
4. Ciphertext, IV, dan salt dikirim ke server
5. Server menyimpan ciphertext ke database

d. Proses Membuka Vault

1. Browser meminta ciphertext ke backend
2. Backend mengirim ciphertext
3. Browser melakukan dekripsi menggunakan AES key
4. Password asli ditampilkan ke user

C. Desain Kriptografi

Berdasarkan implementasi pada file `crypto.js`, proyek SafePass ini menggunakan komponen kriptografi berikut :

Komponen	Algoritma	Parameter	Lokasi Eksekusi
Key Derivation	PBKDF2-SHA-256	Iterasi: 100.000, Output Key: 256 bit	Client (Browser)
Enkripsi Vault	AES-256-GCM	IV/Nonce: 12 byte, Authentication Tag: 128 bit	Client (Browser)
Password Generator	CSPRNG (<code>crypto.getRandomValues</code>)	Panjang Password: 8–24 karakter	Client (Browser)
Hash Password Login	SHA-256	Digest: 256 bit	Client (Browser & Server Verification)
Session Authentication	PHP Session	Session ID berbasis cookie	Server (PHP Backend)
Penyimpanan Data	MySQL Database	Ciphertext, IV, Salt, Metadata Vault	Server
Transport Security	HTTPS/TLS	TLS 1.2+	Client ↔ Server

1. PBKDF2 + SHA-256

a. Algoritma yang Dipilih

SafePass menggunakan PBKDF2 dengan SHA-256 untuk proses key derivation dari master password pengguna. Algoritma ini menghasilkan key AES 256-bit yang digunakan untuk proses enkripsi vault password.

b. Alasan Pemilihan

PBKDF2 dipilih karena mampu memperlambat proses brute force melalui mekanisme iterasi sebanyak 100.000 kali. Selain itu, PBKDF2 kompatibel dengan Web Crypto API sehingga mudah diimplementasikan pada browser. Penggunaan SHA-256 juga memberikan tingkat keamanan hashing yang baik dan masih banyak digunakan pada sistem.

c. Alternatif yang Dipertimbangkan

Alternatif lain yang dipertimbangkan adalah `bcrypt` dan `Argon2`. `bcrypt` memiliki keamanan yang baik untuk password hashing, tetapi implementasinya pada browser lebih terbatas. `Argon2` memiliki tingkat keamanan lebih modern dan tahan terhadap GPU attack, namun dukungan

native pada Web Crypto API masih terbatas dan implementasinya lebih kompleks.

Karena alasan kompatibilitas dan kemudahan implementasi client-side, PBKDF2 dipilih sebagai solusi utama.

2. AES-256-GCM

a. Algoritma yang Dipertimbangkan

SafePass menggunakan AES-256-GCM untuk mengenkripsi data vault password pengguna sebelum dikirim ke server.

b. Alasan Pemilihan

AES-256-GCM dipilih karena memiliki tingkat keamanan tinggi dan mendukung autentikasi data melalui authentication tag. Mode GCM memungkinkan sistem mendeteksi perubahan ciphertext sehingga integritas data tetap terjaga. Selain itu, AES merupakan standar enkripsi modern yang cepat dan efisien pada browser.

c. Alternatif yang Dipertimbangkan

Alternatif lain yang dipertimbangkan adalah DES dan Triple DES. DES tidak dipilih karena ukuran key yang kecil dan sudah tidak aman. Triple DES lebih aman dibanding DES, tetapi memiliki performa lebih lambat dibanding AES.

Karena memiliki keamanan dan performa yang lebih baik, AES-256-GCM dipilih sebagai algoritma utama enkripsi vault. DES tidak dipilih karena ukuran key yang kecil dan sudah tidak aman. Triple DES lebih aman dibanding DES, tetapi memiliki performa lebih lambat dibanding AES.

Karena memiliki keamanan dan performa yang lebih baik, AES-256-GCM dipilih sebagai algoritma utama enkripsi vault.

3. CSPRNG (crypto.getRandomValues)

a. Algoritma yang Dipilih

SafePass menggunakan Cryptographically Secure Pseudo Random Number Generator (CSPRNG) melalui fungsi `crypto.getRandomValues()` untuk menghasilkan salt, IV, dan password acak.

b. Alasan Pemilihan

CSPRNG dipilih karena mampu menghasilkan nilai acak yang aman secara kriptografis dan sulit diprediksi. Fungsi ini juga tersedia secara native pada browser modern sehingga lebih efisien dan aman digunakan.

c. Alternatif yang Dipertimbangkan

Alternatif lain yang dipertimbangkan adalah `Math.random()`. Namun `Math.random()` tidak dirancang untuk kebutuhan keamanan karena

nilai random yang dihasilkan masih dapat diprediksi. Oleh karena itu, `crypto.getRandomValues()` dipilih sebagai solusi utama.

D. Implementasi

1. Implementasi Web Crypto API

Sistem menggunakan Web Crypto API bawaan browser.

Contoh konfigurasi:

```
const ITERATIONS = 100000;  
const KEY_LENGTH = 256;  
const ALGORITHM = "AES-GCM";
```

2. Implementasi Generate Salt

```
function generateSalt() {  
    return crypto.getRandomValues(  
        new Uint8Array(16)  
    );  
}
```

Penjelasan:

- Membuat salt sepanjang 16 byte.
- Menggunakan random generator browser.
- Salt berbeda setiap proses.

3. Implementasi Generate IV

```
function generateIV() {  
    return crypto.getRandomValues(  
        new Uint8Array(12)  
    );  
}
```

Penjelasan:

- Membuat IV sepanjang 12 byte.
- Direkomendasikan untuk AES-GCM.

4. Implementasi PBKDF2

```
function generateIV() {  
    return crypto.getRandomValues(  
        new Uint8Array(12)  
    );  
}  
  
return crypto.subtle.deriveKey(  
    {  
        name: "PBKDF2",  
        salt: salt,  
        iterations: ITERATIONS,  
        hash: "SHA-256"  
    },  
    keyMaterial,  
    {  
        name: ALGORITHM,  
        length: KEY_LENGTH  
    },  
    true,  
    ["encrypt", "decrypt"]  
);
```

Penjelasan:

- Menghasilkan key AES dari master password.
- Menggunakan 100000 iterasi.
- Menggunakan SHA-256.

5. Implementasi Enkripsi

```
const encrypted =  
  await crypto.subtle.encrypt(  
    {  
      name: ALGORITHM,  
      iv: iv  
    },  
    key,  
    encoder.encode(data)  
  );
```

Penjelasan:

- Data plaintext diubah menjadi ciphertext.
- Menggunakan AES-GCM.
- IV digunakan untuk keamanan tambahan.

6. Implementasi Penyimpanan Database

Data yang disimpan:

- ciphertext
- iv
- salt

Server tidak menyimpan plaintext password.

7. Implementasi Session dan Logout

Sistem memiliki fitur auto logout.

Tujuan:

1. Mencegah akses saat user lupa logout.
2. Mengurangi risiko penyalahgunaan komputer.
3. Menjaga keamanan session.

E. Pengujian

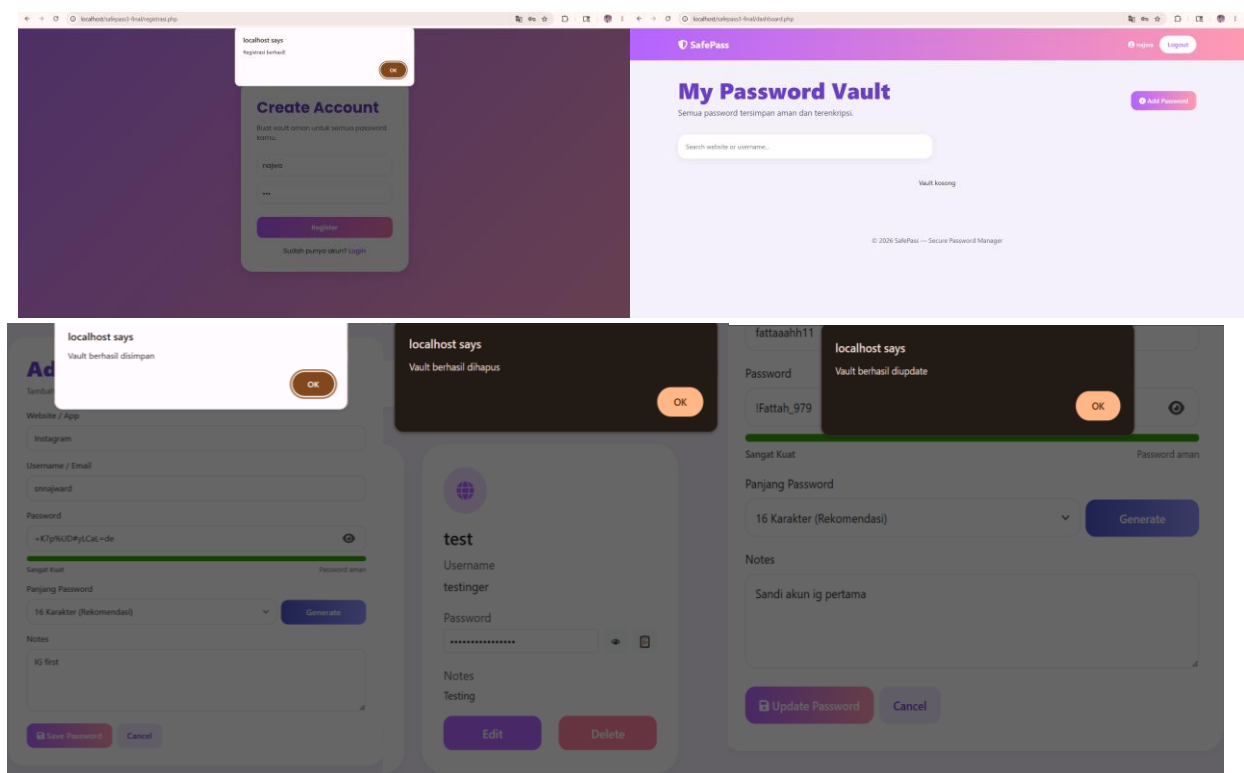
Pengujian berikut disusun dari analisis source code dan dirancang untuk membuktikan fungsi normal, error handling, dan skenario serangan. Fokus pengujian bukan hanya apakah tombol bekerja, tetapi apakah perlindungan keamanan tetap berlaku ketika input salah, session tidak ada, atau data terenkripsi dimodifikasi.

1. Skenario Normal

No.	Skenario	Langkah Uji	Hasil yang Diharapkan
N1	Registrasi user baru	Buka registrasi.php, isi username dan master password, klik Register.	Data masuk ke tabel users, sistem menampilkan alert registrasi berhasil, lalu diarahkan ke login.php.
N2	Login benar	Masukkan username dan master password yang sesuai.	Client menghitung hash yang sama, session PHP dibuat, dan user masuk ke dashboard.php.

N3	Tambah password	Pada add_password.php, isi website, username, password, notes, lalu klik Save Password.	Vault lama didekripsi, entry baru ditambahkan, vault dienkrpsi ulang, dan save_vault.php menyimpan ciphertext baru.
N4	Edit password	Klik Edit pada salah satu card, ubah data, lalu klik Update Password.	edit_password.php memuat data berdasarkan index, vault diperbarui dan dienkrpsi ulang, lalu update_vault.php menyimpan data terbaru.
N5	Hapus password	Klik Delete, lalu konfirmasi penghapusan.	Entry dihapus dari array vault, vault dienkrpsi ulang, dan delete_vault.php menyimpan blob ciphertext baru ke database.

Screenshot pengujian :

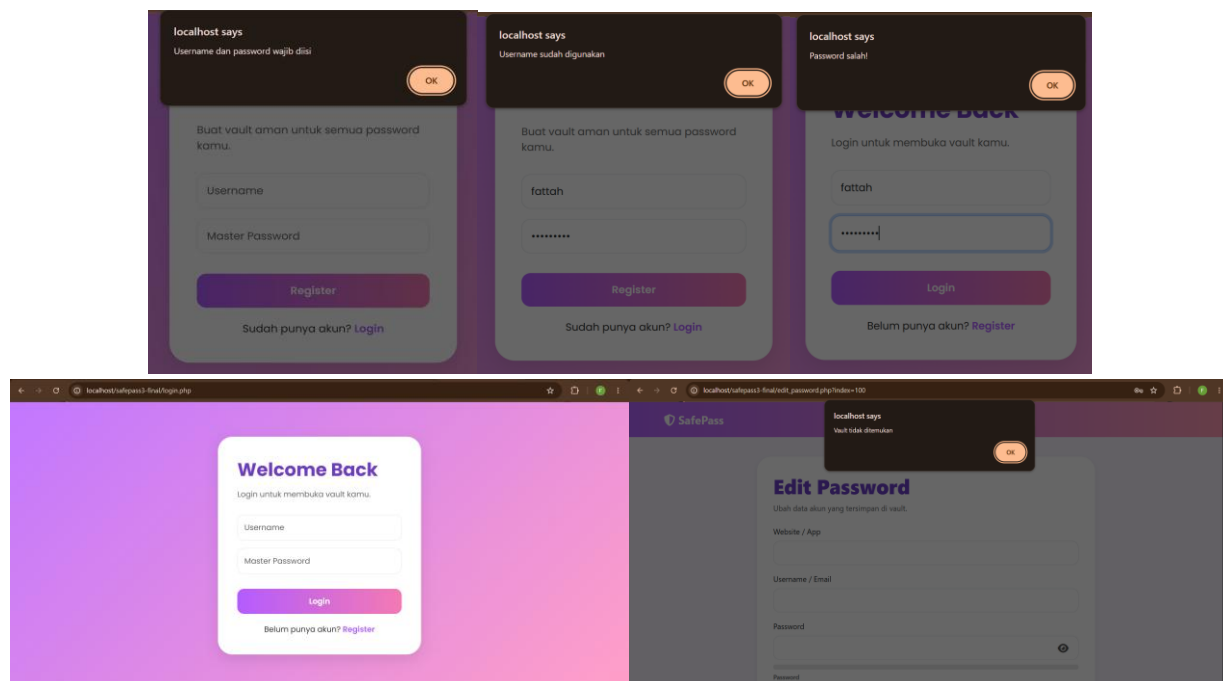


2. Skenario Error

No.	Skenario	Pemicu	Hasil yang Diharapkan / Temuan
E1	Field registrasi kosong	Username atau password tidak diisi saat registrasi.	auth.js menampilkan alert "Username dan password wajib diisi" dan proses registrasi dibatalkan.
E2	Username duplikat	Registrasi menggunakan username yang sudah terdaftar.	register.php memeriksa tabel users dan mengembalikan pesan "Username sudah digunakan".

E3	Password login salah	Master password berbeda dari password saat registrasi.	computedHash tidak sama dengan user.password_hash, sehingga login dihentikan di sisi client.
E4	Akses dashboard tanpa session	User membuka dashboard.php langsung tanpa login.	PHP session guard mendeteksi session tidak valid dan mengarahkan user kembali ke login.php.
E5	Vault kosong	User baru belum pernah menyimpan data vault.	get_vault.php mengembalikan success: false, lalu UI menampilkan informasi "Vault kosong".
E6	Index edit tidak valid	Membuka edit_password.php?index=-1 atau index melebihi jumlah data vault.	Fungsi loadEditVault() menampilkan alert error dan mengembalikan user ke halaman dashboard.

Screenshot pengujian :

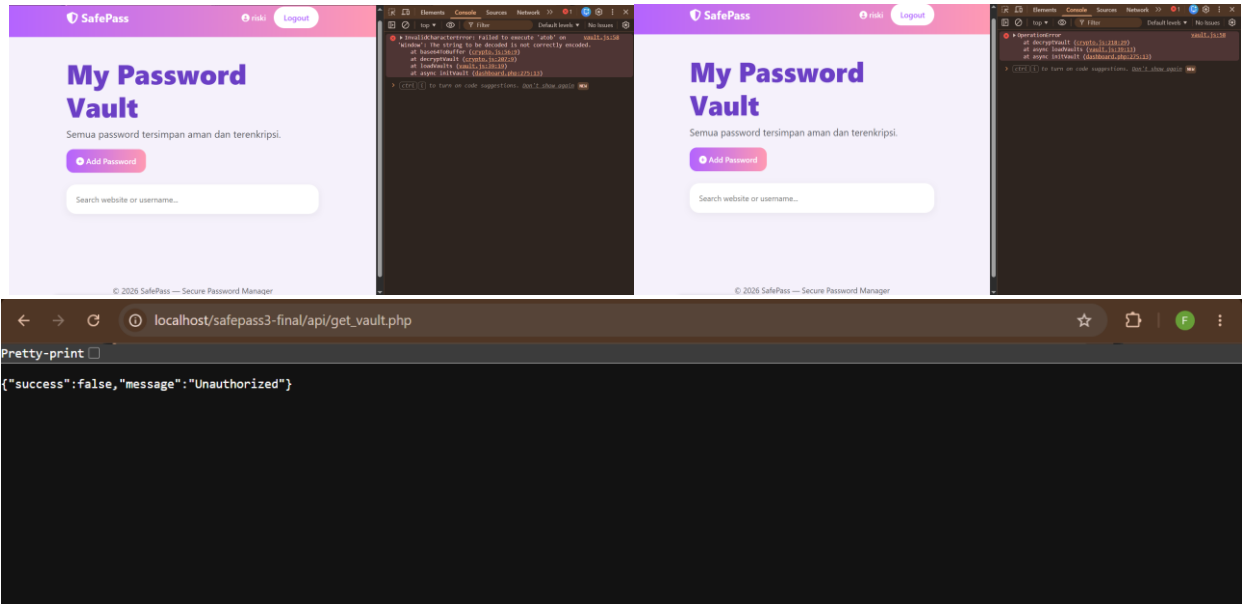


3. Skenario Serangan dan Keamanan

No.	Skenario Serangan	Cara Uji	Ekspektasi	Temuan dari Kode
A1	Tampering ciphertext	Ubah beberapa karakter ciphertext pada database lalu buka vault kembali.	Proses dekripsi AES-256-GCM gagal sehingga vault tidak dapat dibaca.	crypto.subtle.decrypt() akan menghasilkan error apabila authentication tag tidak valid akibat ciphertext dimodifikasi.
A2	Tampering IV	Ubah nilai iv pada database sebelum	Dekripsi gagal karena IV tidak	Fungsi decryptVault() menggunakan IV dari database

		proses decrypt dilakukan.	sesuai dengan data enkripsi awal.	sehingga perubahan IV akan merusak autentikasi AES-GCM.
A3	Akses API vault tanpa session	Kirim request/fetch ke api/get_vault.php tanpa melakukan login terlebih dahulu.	Server menolak request dan mengembalikan status unauthorized.	get_vault.php melakukan pengecekan \$_SESSION["user_id"] sebelum mengakses data vault user.

Screenshot pengujian :



F. Keterbatasan & Refleksi

SafePass sudah menunjukkan pemahaman penting tentang password manager modern: plaintext password tidak boleh disimpan di database, dan proses enkripsi sebaiknya dilakukan sebelum data meninggalkan browser.

Akan tetapi, untuk konteks keamanan nyata, beberapa bagian perlu diperbaiki sebelum aplikasi dianggap aman.

1. **Master Password disimpan di localStorage**, yang mana jika terjadi XSS, attacker bisa membaca master password dan membuka seluruh vault pengguna. **Perbaikan kedepannya:** menyimpan key hanya di memory runtime, dan jika terpaksa gunakan sessionStorage atau minta user memasukan ulang master password setelah refresh browser.
2. **Data user dan vault dirender menggunakan innerHTML tanpa escaping** yang mana Stored XSS bisa terjadi melalui input username, website, maupun notes sehingga script

berbahaya dapat dijalankan di browser korban. **Perbaikan kedepannya:** kami akan menggunakan `textContent` atau `createElement`, dan `escape OUTPUT php` dengan `htmlspecialchars()`. Dan bisa juga aktifin CSP (Content Security Policy).

- 3. Satu bob vault satu user**, yang mana ini kurang efisien untuk data besar dan rawan konflik ketika banyak tab/browser aktif bersamaan. **Perbaikan kedepannya:** menambahkan versioning atau simpan item vault terenkripsi secara terpisah dengan metadata minimal

[Link Video Demonstrasi SafePass](#)